

Inżynieria danych

Wykład 5: organizacja plików

Katarzyna Grygiel

Semestr letni 2025/2026

W idealnym świecie...

Od pamięci komputerowej oczekujemy:

- bardzo dużej pojemności (potencjalnie nieograniczonej),
- natychmiastowego swobodnego dostępu do danych,
- możliwie dużej przepustowości,
- trwałości,
- niskiej ceny.

W idealnym świecie...

Od pamięci komputerowej oczekujemy:

- bardzo dużej pojemności (potencjalnie nieograniczonej),
- natychmiastowego swobodnego dostępu do danych,
- możliwie dużej przepustowości,
- trwałości,
- niskiej ceny.

Żaden z istniejących sposobów przechowywania danych nie spełnia wszystkich powyższych wymagań.

W idealnym świecie...

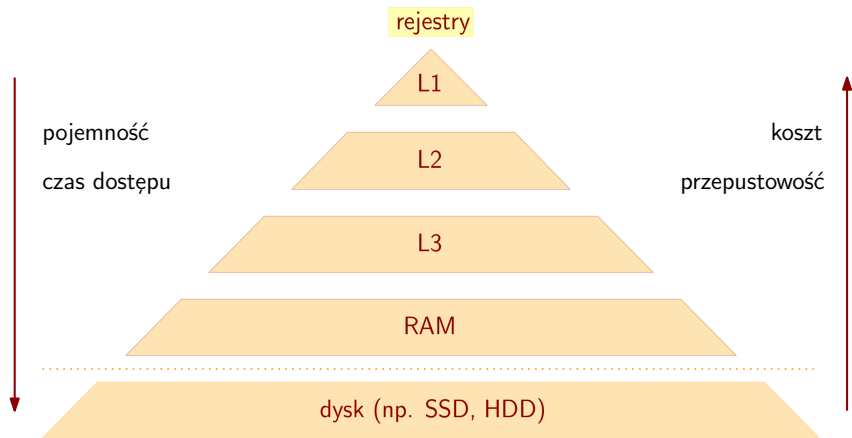
Od pamięci komputerowej oczekujemy:

- bardzo dużej pojemności (potencjalnie nieograniczonej),
- natychmiastowego swobodnego dostępu do danych,
- możliwie dużej przepustowości,
- trwałości,
- niskiej ceny.

Żaden z istniejących sposobów przechowywania danych nie spełnia wszystkich powyższych wymagań.

Musimy zgodzić się na kompromis. Dane używane aktualnie trzymane są w szybszej pamięci, natomiast w wolniejszej te, które nie są aktualnie wykorzystywane.

Hierarchia pamięci



Hierarchia pamięci

- Rejestry: obecnie mają 32 lub 64 bity, stanowią najszybszą formę pamięci, umiejscowione są wewnątrz procesora.

Hierarchia pamięci

- Rejestry: obecnie mają 32 lub 64 bity, stanowią najszybszą formę pamięci, umiejscowione są wewnątrz procesora.
- Pamięć podręczna procesora (cache L1, L2, L3): również jest częścią procesora, przyspiesza dostęp do pamięci głównej.

Hierarchia pamięci

- Rejestry: obecnie mają 32 lub 64 bity, stanowią najszybszą formę pamięci, umiejscowione są wewnątrz procesora.
- Pamięć podręczna procesora (cache L1, L2, L3): również jest częścią procesora, przyspiesza dostęp do pamięci głównej.
- Pamięć główna: znajduje się na płycie głównej, połączona jest z procesorem magistralami, tak jak rejestry i cache jest pamięcią ulotną.

Hierarchia pamięci

- Rejestry: obecnie mają 32 lub 64 bity, stanowią najszybszą formę pamięci, umiejscowione są wewnątrz procesora.
- Pamięć podręczna procesora (cache L1, L2, L3): również jest częścią procesora, przyspiesza dostęp do pamięci głównej.
- Pamięć główna: znajduje się na płycie głównej, połączona jest z procesorem magistralami, tak jak rejestry i cache jest pamięcią ulotną.
- Dysk twardy (HDD) lub półprzewodnikowy (SSD): nie jest podłączony bezpośrednio do procesora, jest za to pamięcią trwałą; odczyt z dysku odbywa się przez przesłanie danych do odpowiednich bloków w pamięci RAM.

Hierarchia pamięci

- Rejestry: obecnie mają 32 lub 64 bity, stanowią najszybszą formę pamięci, umiejscowione są wewnątrz procesora.
- Pamięć podręczna procesora (cache L1, L2, L3): również jest częścią procesora, przyspiesza dostęp do pamięci głównej.
- Pamięć główna: znajduje się na płycie głównej, połączona jest z procesorem magistralami, tak jak rejestry i cache jest pamięcią ulotną.
- Dysk twardy (HDD) lub półprzewodnikowy (SSD): nie jest podłączony bezpośrednio do procesora, jest za to pamięcią trwałą; odczyt z dysku odbywa się przez przesłanie danych do odpowiednich bloków w pamięci RAM.
- Dyski optyczne (CD, DVD): czas odczytu jest znacznie dłuższy niż dla dysków HDD.

Hierarchia pamięci

- Rejestry: obecnie mają 32 lub 64 bity, stanowią najszybszą formę pamięci, umiejscowione są wewnątrz procesora.
- Pamięć podręczna procesora (cache L1, L2, L3): również jest częścią procesora, przyspiesza dostęp do pamięci głównej.
- Pamięć główna: znajduje się na płycie głównej, połączona jest z procesorem magistralami, tak jak rejestry i cache jest pamięcią ulotną.
- Dysk twardy (HDD) lub półprzewodnikowy (SSD): nie jest podłączony bezpośrednio do procesora, jest za to pamięcią trwałą; odczyt z dysku odbywa się przez przesłanie danych do odpowiednich bloków w pamięci RAM.
- Dyski optyczne (CD, DVD): czas odczytu jest znacznie dłuższy niż dla dysków HDD.
- Pamięć taśmowa: najtrwalsza i najbardziej pojemna, jednak o długim czasie dostępu; służy najczęściej do archiwizacji.

Operacje między pamięcią wewnętrzną a zewnętrzną

Większość systemów zarządzania bazą danych przechowuje dane na dyskach twardych.

Wykonanie operacji na danych wymaga uprzedniego zapisania ich w pamięci RAM, wykonania operacji tamże, a następnie przepisania wyniku na dysk.

Operacje między pamięcią wewnętrzną a zewnętrzną

Większość systemów zarządzania bazą danych przechowuje dane na dyskach twardych.

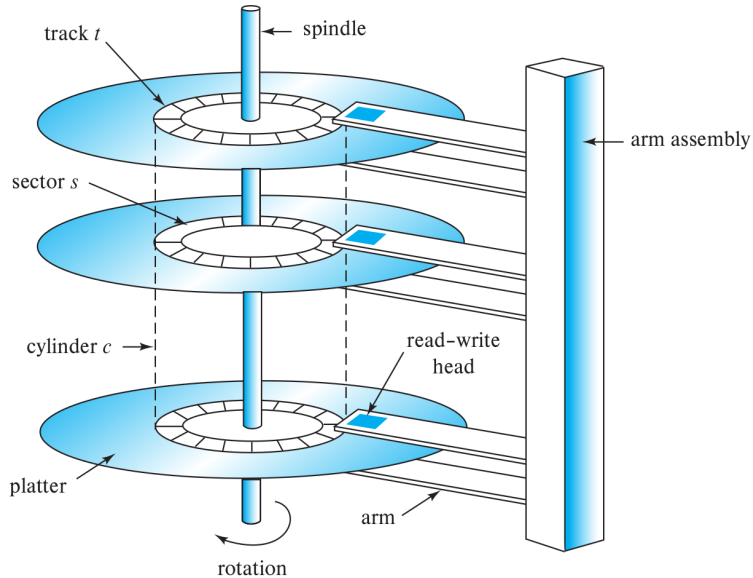
Wykonanie operacji na danych wymaga uprzedniego zapisania ich w pamięci RAM, wykonania operacji tamże, a następnie przepisania wyniku na dysk.

Oznacza to konieczność przeprowadzenia następujących operacji:

- READ – przesłanie danych z dysku do pamięci RAM,
- WRITE – przesłanie danych z pamięci RAM na dysk.

Ponieważ obie operacje są znacznie wolniejsze niż operacje w pamięci RAM, należy je wykonywać umiejętnie. Koszt operacji w bazie danych jest przedstawiany jako liczba powyższych operacji.

Budowa i działanie dysku twardego



Czas dostępu do danych

Każde żądanie odczytu lub zapisu zawiera adres na dysku, który jest podany w postaci numeru bloku dyskowego (obejmującego jeden lub kilka sektorów). Rozmiar takiego bloku waha się obecnie od 4 do 16 kB. Dane są przesyłane pomiędzy dyskiem a pamięcią główną w blokach.

Czas dostępu do danych to czas od momentu wysłania żądania odczytu/zapisu do momentu transferu danych.

Czas dostępu do danych

Każde żądanie odczytu lub zapisu zawiera adres na dysku, który jest podany w postaci numeru bloku dyskowego (obejmującego jeden lub kilka sektorów). Rozmiar takiego bloku waha się obecnie od 4 do 16 kB. Dane są przesyłane pomiędzy dyskiem a pamięcią główną w blokach.

Czas dostępu do danych to czas od momentu wysłania żądania odczytu/zapisu do momentu transferu danych.

Ramię ustawia się najpierw nad właściwą ścieżką, następnie głowica musi znaleźć się nad odpowiednim sektorem, aż w końcu następuje transfer danych. Zatem na czas dostępu składają się:

- czas przeszukiwania (ruch ramienia),
- opóźnienie ze względu na rotację (ustawienie właściwego kąta),
- czas przesyłu danych.

Czas dostępu do danych

Ponieważ czas dostępu do danych na dysku zależy od ich położenia na dysku, wzajemne rozmieszczenie bloków może mieć zasadniczy wpływ na szybkość działania systemu bazodanowego. Najlepsze wyniki uzyskuje się dla ciągów sąsiadujących ze sobą bloków.

Czas dostępu do danych

Ponieważ czas dostępu do danych na dysku zależy od ich położenia na dysku, wzajemne rozmieszczenie bloków może mieć zasadniczy wpływ na szybkość działania systemu bazodanowego. Najlepsze wyniki uzyskuje się dla ciągów sąsiadujących ze sobą bloków.

Dobrym rozwiązaniem jest przechowywanie danych, które są często wykorzystywane przez programy aplikacyjne, na stałe w buforach pamięci RAM. Dostęp do nich jest wtedy bardzo szybki.

Czas dostępu do danych

Ponieważ czas dostępu do danych na dysku zależy od ich położenia na dysku, wzajemne rozmieszczenie bloków może mieć zasadniczy wpływ na szybkość działania systemu bazodanowego. Najlepsze wyniki uzyskuje się dla ciągów sąsiadujących ze sobą bloków.

Dobrym rozwiązaniem jest przechowywanie danych, które są często wykorzystywane przez programy aplikacyjne, na stałe w buforach pamięci RAM. Dostęp do nich jest wtedy bardzo szybki.

Operacje odczytu i zapisu bloków na dysku mogą być realizowane współbieżnie, dlatego transakcje użytkowników są realizowane przez system współbieżnie, a nie sekwencyjnie. Również w ramach jednej transakcji rozłożenie danych do różnych dysków może istotnie przyspieszyć jej wykonanie.

Przechowywanie bazy danych

Baza danych najczęściej przechowywana jest w wielu plikach, które zlokalizowane są na stałe na dysku.

Każdy plik jest podzielony na jednostki logiczne stałej długości zwane stronami. Rozmiar stron jest często określony przez SZBD (np. w PostgreSQL domyślny rozmiar strony to 8 kB).

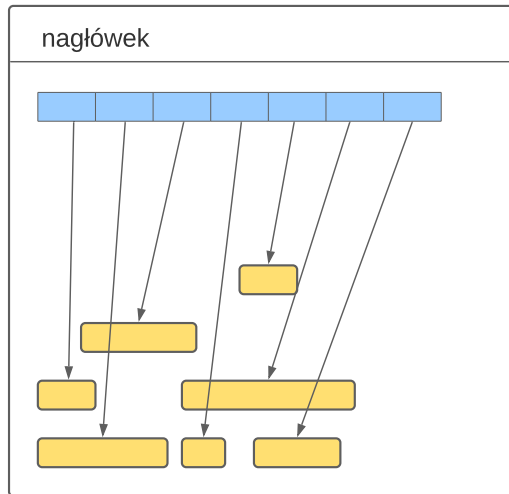
Przechowywanie bazy danych

Baza danych najczęściej przechowywana jest w wielu plikach, które zlokalizowane są na stałe na dysku.

Każdy plik jest podzielony na jednostki logiczne stałej długości zwane stronami. Rozmiar stron jest często określony przez SZBD (np. w PostgreSQL domyślny rozmiar strony to 8 kB).

Poszczególne relacje przechowujemy w osobnych plikach. Każda krotka powinna zostać zapisana w całości tylko na jednej stronie. Jeśli rozmiar krotki jest większy niż rozmiar strony, to dzieli się ją na części i przechowuje je na osobnych (najlepiej sąsiadujących) stronach. Duże obiekty (LOB) trzymane są w osobnych obszarach, natomiast w pliku dla danej relacji przechowuje się tylko ich lokalizatory.

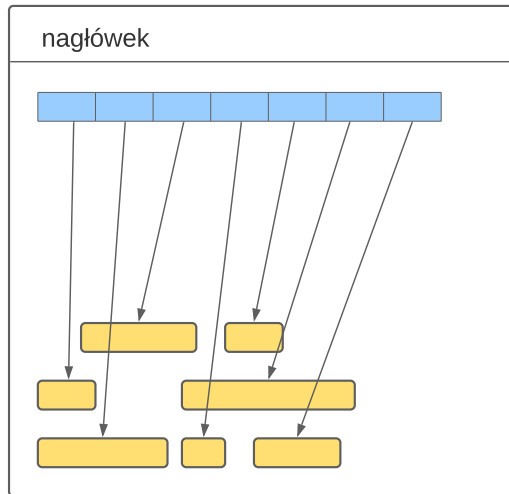
Zapis krotek na stronie ze slotami



Na stronie ze slotami wyróżniamy trzy regiony:

- nagłówek z metadanymi,
- tablica z adresami (sloty),
- właściwe dane (krotki).

Zapis krotek na stronie ze slotami



Na stronie ze slotami wyróżniamy trzy regiony:

- nagłówek z metadanymi,
- tablica z adresami (sloty),
- właściwe dane (krotki).

Zapis krotek na stronie ze slotami

Slot odpowiadający danej krotce zawiera informację, w którym miejscu znajduje się pierwszy bajt zapisanej krotki (`offset`) i jaki jest jej rozmiar (`length`).

Krotki mają adresy postaci

`(nr_strony, nr_slotu)`.

Zapis krotek na stronie ze slotami

Slot odpowiadający danej krotce zawiera informację, w którym miejscu znajduje się pierwszy bajt zapisanej krotki (`offset`) i jaki jest jej rozmiar (`length`).

Krotki mają adresy postaci

`(nr_strony, nr_slotu)`.

Adresy odwołują się do pamięci wirtualnej. Oznacza to, że w pamięci fizycznej adres krotki może się zmienić, ale z punktu widzenia użytkownika adres pozostaje niezmienny. Dzięki temu nie trzeba modyfikować referencji do danej krotki.

Strony ze slotami

Strona zaczyna się nagłówkiem, po nim zapisane są sloty, natomiast dane zapisywane są od końca strony. Strona jest pełna, gdy nie można już dopisać nowej krotki.

W praktyce SZBD rezerwuje miejsce w każdym bloku na ewentualne modyfikacje zawartych w nim krotek.

Strony ze slotami

Strona zaczyna się nagłówkiem, po nim zapisane są sloty, natomiast dane zapisywane są od końca strony. Strona jest pełna, gdy nie można już dopisać nowej krotki.

W praktyce SZBD rezerwuje miejsce w każdym bloku na ewentualne modyfikacje zawartych w nim krotek.

Usuwanie krotki polega na usunięciu referencji w odpowiadającym jej slotcie. Po wstawieniu nowej krotki, nowy slot zostaje dopisany na końcu tablicy.

Strony ze slotami

Strona zaczyna się nagłówkiem, po nim zapisane są sloty, natomiast dane zapisywane są od końca strony. Strona jest pełna, gdy nie można już dopisać nowej krotki.

W praktyce SZBD rezerwuje miejsce w każdym bloku na ewentualne modyfikacje zawartych w nim krotek.

Usuwanie krotki polega na usunięciu referencji w odpowiadającym jej slotcie. Po wstawieniu nowej krotki, nowy slot zostaje dopisany na końcu tablicy.

Jeśli krotka znajdzie się na innej stronie, to można na starej stronie podać referencję do nowej strony. Pozwala to użytkownikom korzystać ze starych adresów (oczywiście kosztem spowolnienia operacji). W praktyce nie tworzy się więcej niż jednej referencji do innej strony.

Przykład w PostgreSQL

W PostgreSQL adres krotki można uzyskać przez ctid.
Czyszczenie stron dla tabeli *tab* wykonuje się poleceniem

```
VACUUM FULL tab;
```

demo

```
CREATE TABLE tab (n INT);  
INSERT INTO tab VALUES (1), (2), (3), (4);  
SELECT ctid, * FROM tab;  
DELETE FROM tab WHERE mod(n, 2) = 0;  
SELECT ctid, * FROM tab;  
INSERT INTO tab VALUES (5), (6);  
SELECT ctid, * FROM tab;  
VACUUM FULL tab;  
SELECT ctid, * FROM tab;
```

Sloty: krotki stałej długości

Rozważmy krotki relacji *studenci*(id, nazwisko, grupa) typu

`(int, char(40), char(3))`.

Zakładając, że `int` ma rozmiar 4 B, natomiast `char` 1 B, każda krotka jest rozmiaru 47 B.

Dla strony o rozmiarze 4 kB offset krotki adresowanej w k -tym slotcie wynosi

$$\text{offset}(k) = 4096 - 47(k + 1).$$

Sloty: krotki stałej długości

Rozważmy krotki relacji *studenci* (id, nazwisko, grupa) typu

`(int, char(40), char(3))`.

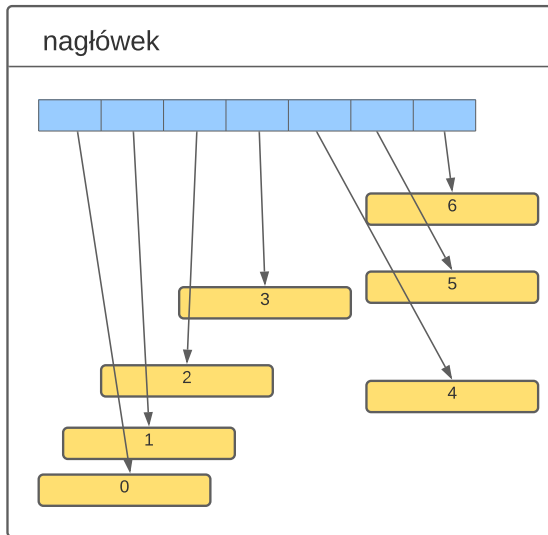
Zakładając, że `int` ma rozmiar 4 B, natomiast `char` 1 B, każda krotka jest rozmiaru 47 B.

Dla strony o rozmiarze 4 kB offset krotki adresowanej w k -tym slotcie wynosi

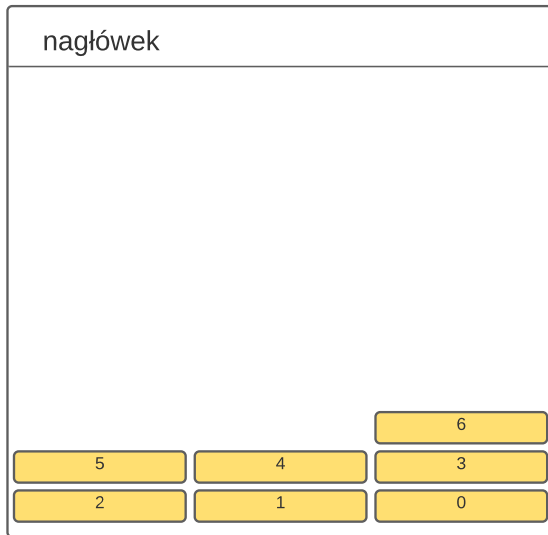
$$\text{offset}(k) = 4096 - 47(k + 1).$$

Zatem w tym przypadku nie trzeba przechowywać informacji o początku krotki. Tym samym w przypadku krotek stałej długości nie trzeba korzystać ze slotów.

Sloty: krotki stałej długości



Sloty: krotki stałej długości



Sloty: krotki zmiennej długości

Rozważmy krotki relacji *studenci*(id, nazwisko, grupa) typu

(int, varchar, char(3)).

Tym razem rozmiar krotki jest zmienny. Można przechowywać dane dla atrybutów o stałej długości (id oraz grupa) oraz metainformacje o danych dla atrybutów zmiennej długości (nazwisko) jako rekord

(id, grupa, nazwisko_offset, nazwisko_length)

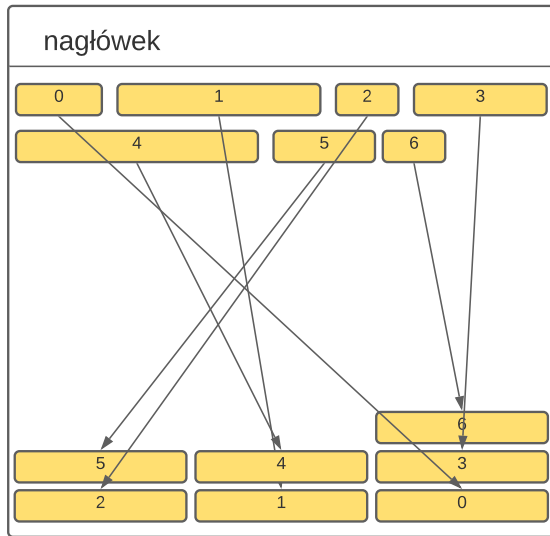
typu

(int, char(3), smallint, smallint)

o długości $4 + 3 + 2 + 2 = 11$ bajtów na końcu strony, natomiast pozostałe dane (czyli właściwe wartości atrybutu nazwisko) zapisać zaraz po nagłówku strony. Wtedy dla k -tej krotki mamy

$$\text{offset}(k) = 4096 - 11(k + 1).$$

Sloty: krotki zmiennej długości



Układ krotek na stronie

Relacja, jako zbiór krotek, może być postrzegana jako obiekt dwuwymiarowy. Chcąc ją przechować, musimy ją zlinearyzować, czyli przedstawić jako jednowymiarowy ciąg danych.

Możemy krotki danej relacji zapisać jedna po drugiej, ale poza takim układem wierszowym możliwych jest wiele innych układów, np. kolumnowy lub hybrydowy.

Wybór układu zależy od tego, jakie operacje są najczęściej wykonywane. W szczególności decyduje cel wykorzystania systemu bazodanowego: czy ma on charakter transakcyjny, czy też używany jest do analizy danych.

Linearyzacja krotek: układ wierszowy

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w układzie wierszowym wyglądają następująco:

1, Zofia, 2a, 2, Piotr, 7b, 3, Jan, 11c

Linearyzacja krotek: układ wierszowy

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w układzie wierszowym wyglądają następująco:

1, Zofia, 2a, 2, Piotr, 7b, 3, Jan, 11c

Przykładowe zapytania, gdy ten układ sprawdza się bardzo dobrze:

- `SELECT * FROM studenci;`
- `SELECT * FROM studenci WHERE id = 2;`
- `INSERT INTO studenci VALUES (4, 'Ewa', '2a');`
- `DELETE FROM studenci WHERE grupa = '2a';`

Linearyzacja krotek: układ wierszowy

Zalety:

- łatwy w implementacji,
- dobrze się sprawdza przy wielu transakcjach,
- wygodny w przypadku dostępu do pojedynczych wierszy.

Wady:

- niewygodny w analizie danych,
- nie sprawdza się dla szerokich tabel (tj. o wielu atrybutach).

Linearyzacja krotek: układ kolumnowy (skorelowany)

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w skorelowanym układzie kolumnowym wyglądają następująco:

1, 2, 3, Zofia, Piotr, Jan, 2a, 7b, 11c

Linearyzacja krotek: układ kolumnowy (skorelowany)

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w skorelowanym układzie kolumnowym wyglądają następująco:

1, 2, 3, Zofia, Piotr, Jan, 2a, 7b, 11c

Przykładowe zapytania, gdy ten układ sprawdza się bardzo dobrze:

- `SELECT imię FROM studenci WHERE grupa = '2a';`
- `ALTER TABLE studenci ADD nazwisko VARCHAR(20);`
- `ALTER TABLE studenci DROP imię;`
- `SELECT AVG(id) FROM studenci;`

Linearyzacja krotek: układ kolumnowy (skorelowany)

Zalety:

- wygodny w przypadku dostępu do niewielkiej liczby kolumn,
- dobrze się sprawdza w analizie danych.

Wady:

- niewygodny w przypadku dostępu do wielu atrybutów,
- nie sprawdza się przy wielu transakcjach.

Linearyzacja krotek: układ kolumnowy (nieskorelowany)

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w nieskorelowanym układzie kolumnowym mogą wyglądać następująco:

1, 3, 2, Jan, Zofia, Piotr, 7b, 11c, 2a

Linearyzacja krotek: układ kolumnowy (nieskorelowany)

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w nieskorelowanym układzie kolumnowym mogą wyglądać następująco:

1, 3, 2, Jan, Zofia, Piotr, 7b, 11c, 2a

W tym przypadku trzeba dodatkowo przechowywać klucze dla poszczególnych wartości, aby móc odtworzyć całą krotkę. Brak porządku powoduje również wolniejsze wykonywanie zapytań wymagających dostępu do kilku atrybutów.

Linearyzacja krotek: układ kolumnowy (nieskorelowany)

<i>studenci</i>		
id	imię	grupa
1	Zofia	2a
2	Piotr	7b
3	Jan	11c

Powyższe krotki zapisane w nieskorelowanym układzie kolumnowym mogą wyglądać następująco:

1, 3, 2, Jan, Zofia, Piotr, 7b, 11c, 2a

W tym przypadku trzeba dodatkowo przechowywać klucze dla poszczególnych wartości, aby móc odtworzyć całą krotkę. Brak porządku powoduje również wolniejsze wykonywanie zapytań wymagających dostępu do kilku atrybutów.

Taki układ ma jednak pewne zalety: nie trzeba przejmować się kolejnością danych i dla ustalonego atrybutu pomiędzy istniejące dane można wstawiać nowe wartości.

Inne układy

Jaką decyzję podjąć, jeśli często pojawiają się zapytania zarówno typu

```
SELECT * FROM studenci WHERE id > 2;
```

jak i

```
SELECT AVG(id) FROM studenci GROUP BY grupa;
```

Można przechowywać dane równocześnie na dwa sposoby i korzystać z tego układu, który lepiej odpowiada danemu zapytaniu. Oczywiście zwiększa to wymagania pamięciowe, jak również czasowe (przy modyfikowaniu danych).

Inne układy

Jaką decyzję podjąć, jeśli często pojawiają się zapytania zarówno typu

```
SELECT * FROM studenci WHERE id > 2;
```

jak i

```
SELECT AVG(id) FROM studenci GROUP BY grupa;
```

Można przechowywać dane równocześnie na dwa sposoby i korzystać z tego układu, który lepiej odpowiada danemu zapytaniu. Oczywiście zwiększa to wymagania pamięciowe, jak również czasowe (przy modyfikowaniu danych).

Na podstawie historii zapytań można dostosować układ (np. stworzyć układ hybrydowy) tak, aby najlepiej odpowiadał przeciętnemu zapytaniu.